Machine Learning Model Deployment Report

Seneca AIG200 – Capstone Project

Machine learning Model Deployment Assignment

Student Information:

Maminirina Fabien Rakotoson

117328245

Date: Feb 18 2026

Executive Summary

This project involves the deployment of a **sentiment analysis model** living on a deployed REST API. The project utilizes a dataset of 31,000 labeled sentences (negative, neutral, and positive) from Kaggle. This dataset will be used to train and evaluate the performance of two models: (1) a traditional TF-IDF + Logistic Regression approach and (2) a fine-tuned DistilBERT transformer model.

The second model performed the best and was wrapped in a Docker image together with the FastAPI application. The Docker image was pushed to Docker Hub and hosted on AWS Lightsail Container Service with API key protection.

Machine Learning Problem & Dataset

Problem to solve:

Given an input sentence, predict its sentiment class: negative, neutral, or positive.

Dataset:

Link: https://www.kaggle.com/datasets/mabubakrsiddiq/sentiment-analysis-dataset-31k-samples

Description: Contains 31,000 text samples, labeled with the sentiments *neutral, negative, or positive*.

The dataset was downloaded locally and stored in the GitHub repository as sentiment_dataset.csv. It originally contains four columns (id, text, label, sentiment), but only the text and sentiment label were used for training.

Train/Validation/Test split:

The data was split using train_test_split with stratification into 72% training, 8% validation, and 20% testing.

Preprocessing pipeline

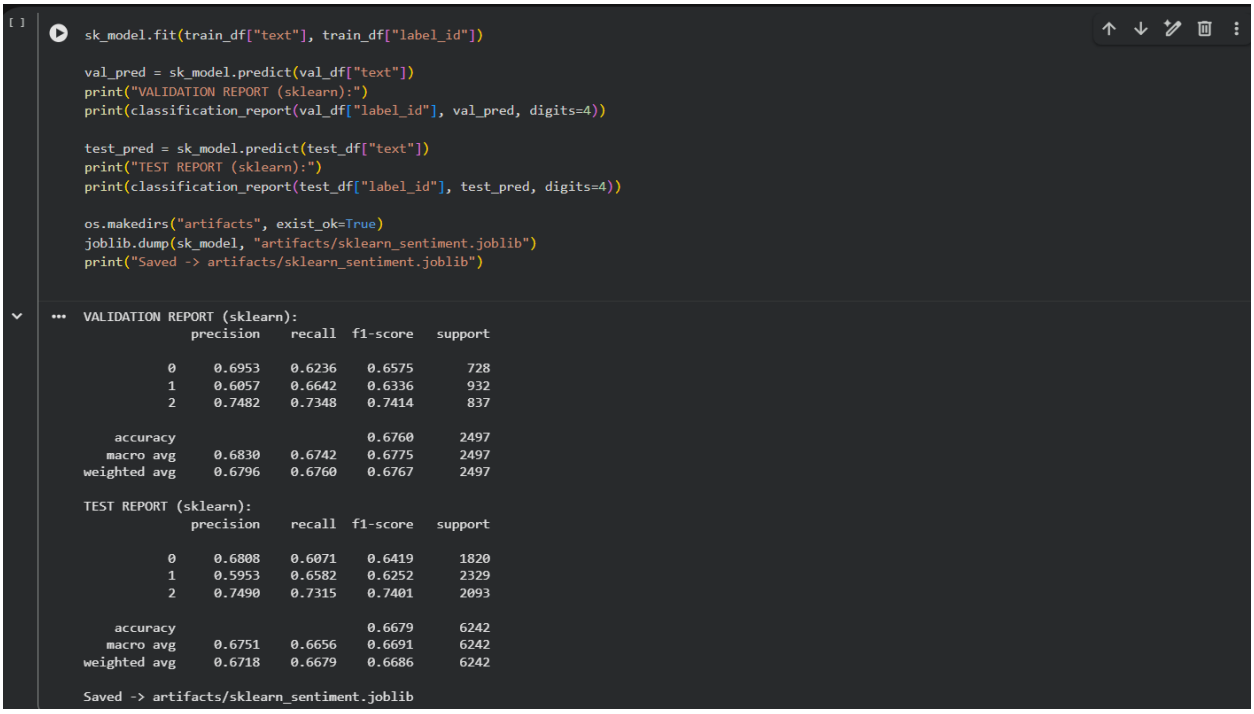
For preprocessing, these were the taken steps:

- Inspect the dataset, check for missing values, remove duplicates, and keep only the required columns.
- Split the dataset using train_test_split with stratified sampling (72% train, 8% validation, 20% test).

Training (Overview)

1) 1st Model: TF-IDF + Logistic Regression

- First, I trained a simple baseline with TF-IDF features and a Logistic Regression classifier. The goal is to: convert text into n-gram features (so the model sees single words + short phrases), then let Logistic Regression learn the sentiment boundaries.
- Stated TF-IDF hyperparameters
    - Ngram_range=(1,2) ➔ (unigrams + bigrams)
    - Max_features=200000 ➔ only 200k most frequent n-gram kept
- Stated LogisticRegression hyperparameters
    - Max_iter=2000
- Performance:
    - Accuracy = 0.6679
    - F1-score = 0.6686



```
sk_model.fit(train_df["text"], train_df["label_id"])

val_pred = sk_model.predict(val_df["text"])
print("VALIDATION REPORT (sklearn):")
print(classification_report(val_df["label_id"], val_pred, digits=4))

test_pred = sk_model.predict(test_df["text"])
print("TEST REPORT (sklearn):")
print(classification_report(test_df["label_id"], test_pred, digits=4))

os.makedirs("artifacts", exist_ok=True)
joblib.dump(sk_model, "artifacts/sklearn_sentiment.joblib")
print("Saved -> artifacts/sklearn_sentiment.joblib")
```

```
VALIDATION REPORT (sklearn):
              precision    recall  f1-score   support

           0     0.6953    0.6236    0.6575       728
           1     0.6057    0.6642    0.6336       932
           2     0.7482    0.7348    0.7414       837

    accuracy                         0.6760      2497
   macro avg     0.6830    0.6742    0.6775      2497
weighted avg     0.6796    0.6760    0.6767      2497

TEST REPORT (sklearn):
              precision    recall  f1-score   support

           0     0.6808    0.6071    0.6419      1820
           1     0.5953    0.6582    0.6252      2329
           2     0.7490    0.7315    0.7401      2093

    accuracy                         0.6679      6242
   macro avg     0.6751    0.6656    0.6691      6242
weighted avg     0.6718    0.6679    0.6686      6242

Saved -> artifacts/sklearn_sentiment.joblib
```

Figure 1: 1st Model Evaluation

2) 2nd Model: Fine-tuned DistilBERT

We then fine-tuned a transformer model for better performance on the sentiment classification task.

- Model used: distilbert-base-uncased
- Steps
    - Convert the pandas' dataset to Hugging Face dataset

- Tokenize the text using the AutoTokenizer.from_pretrained("distilbert-base-uncased")
- Use Trainer to run the training loop, with the dataset, the selected model, and various hyperparameters (epochs, leaning_rate, metrics, ...)
- Evaluate the model

- Performance
  - Accuracy: 0.7577
  - F1-score: 0.758

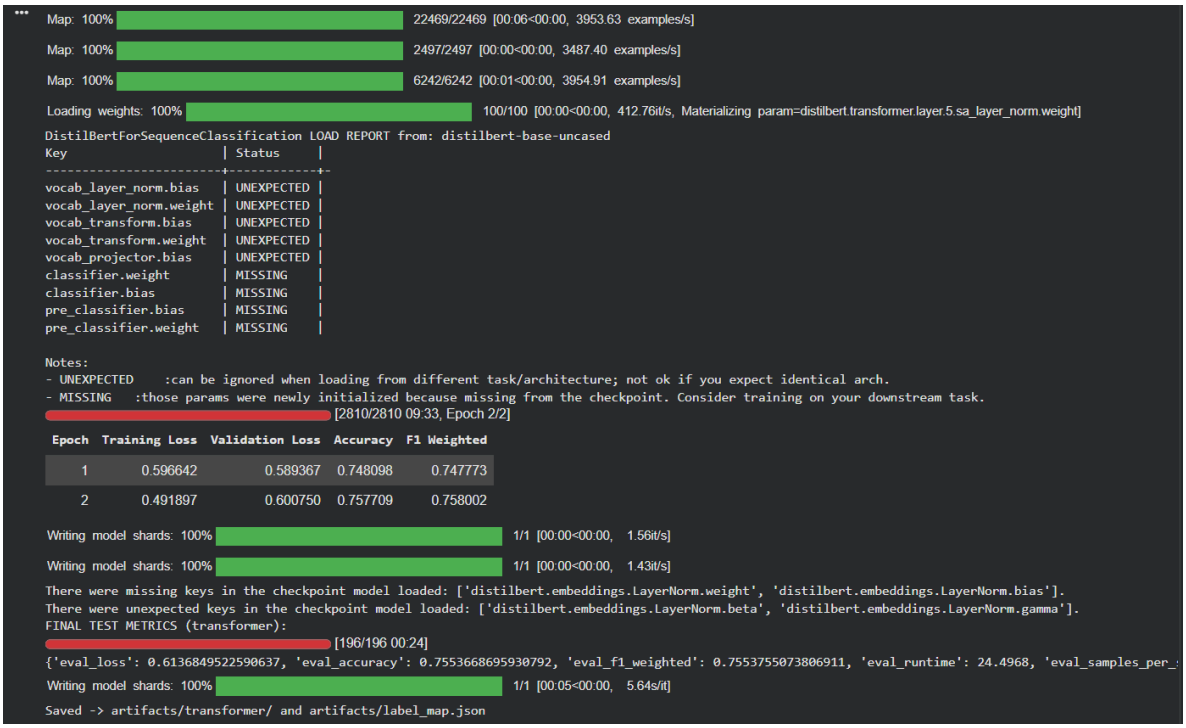


Figure 2: 2nd Model Evaluation

Deployment Architecture

This API is deployed as a Docker container on AWS Lightsail Container Service. The container image is built locally from the project's Dockerfile, pushed to Docker Hub, and then pulled by Lightsail during deployment. Once deployed, user can use the model by calling an endpoint exposed by the deployed Lightsail Container.

Main components

- Client: Browser / Postman / curl (sends requests to the deployed API).
- Docker Hub (public registry): stores the container image (e.g., rakotosonfabien/sentiment-api:latest).
- AWS Lightsail Container Service: pulls the image from Docker Hub and runs it as a managed container.
- Container internals: Uvicorn + FastAPI application, which loads the sentiment model and handles inference requests.

Implementation

The deployment was done in four steps:

1) Build the Docker image locally
- Run the Docker image locally using the Dockerfile in the local repository.
- The FastAPI app + model code are packaged using the project Dockerfile.
- This produces a runnable container image.

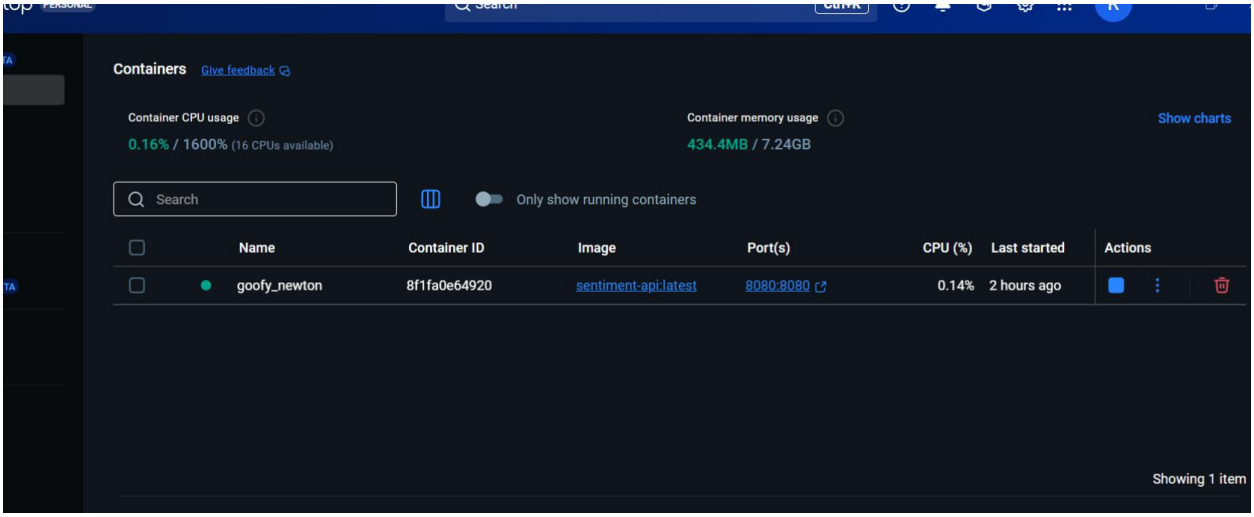


Figure 3: Local running Docker image

2) Push the image to Docker Hub
- After building, the image is uploaded to Docker Hub so AWS can pull it later.

- That can be done by pushing the docker image to Docker Hub using my user account. And will be tagged as `rakotosonfabien/sentiment-api:latest`.

```
l-deployment>docker push rakotosonfabien/sentiment-api:latest
The push refers to repository [docker.io/rakotosonfabien/sentiment-api]
b01ebb1eca54: Pushing  1.362GB/4.265GB
d85099f0969e: Pushed
d815caa91e6c: Pushed
0c8d55a45c0d: Pushed
804fbc89f972: Pushed
8a1de3071079: Pushed
389a21bd537e: Pushed
8cbc47ff628d: Pushed
64faa99400e1: Pushed
e7289ba2d714: Pushed
```

Figure 4: Pushing Docker Image

3) Deploy the image on AWS Lightsail Container Service

- In Lightsail Containers, a deployment is created using the Docker Hub image.
- Lightsail pulls the image and starts the container as a managed service.

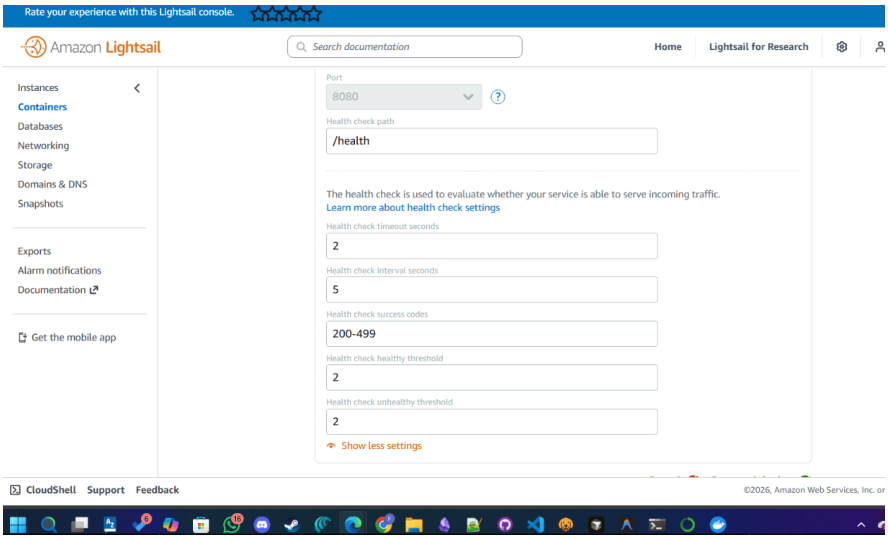


Figure 5: LightSail Container Service Setup

4) Expose the service port and configure health checks

- The container listens on port 8080, and Lightsail exposes it publicly.
- A health check is configured with path “/health”
- Users can use call the API endpoints through curl command, or different tools (Postman, Bruno, ...)

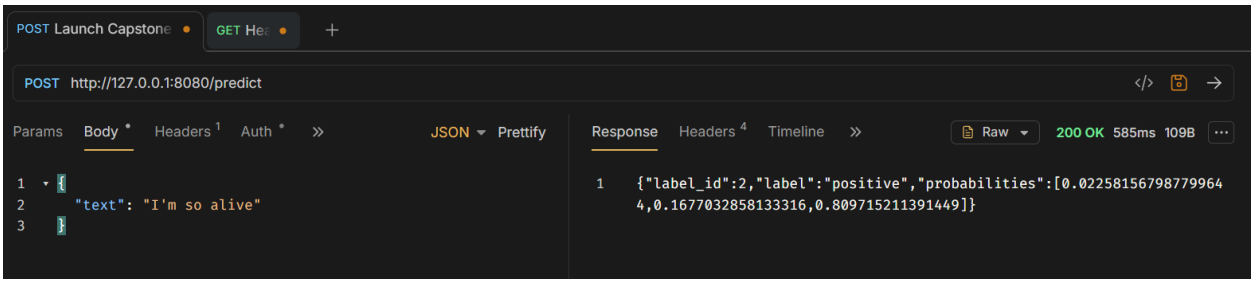


Figure 6: API Call using Bruno

The figure below (Figure 7) shows the architecture diagram for the deployment.

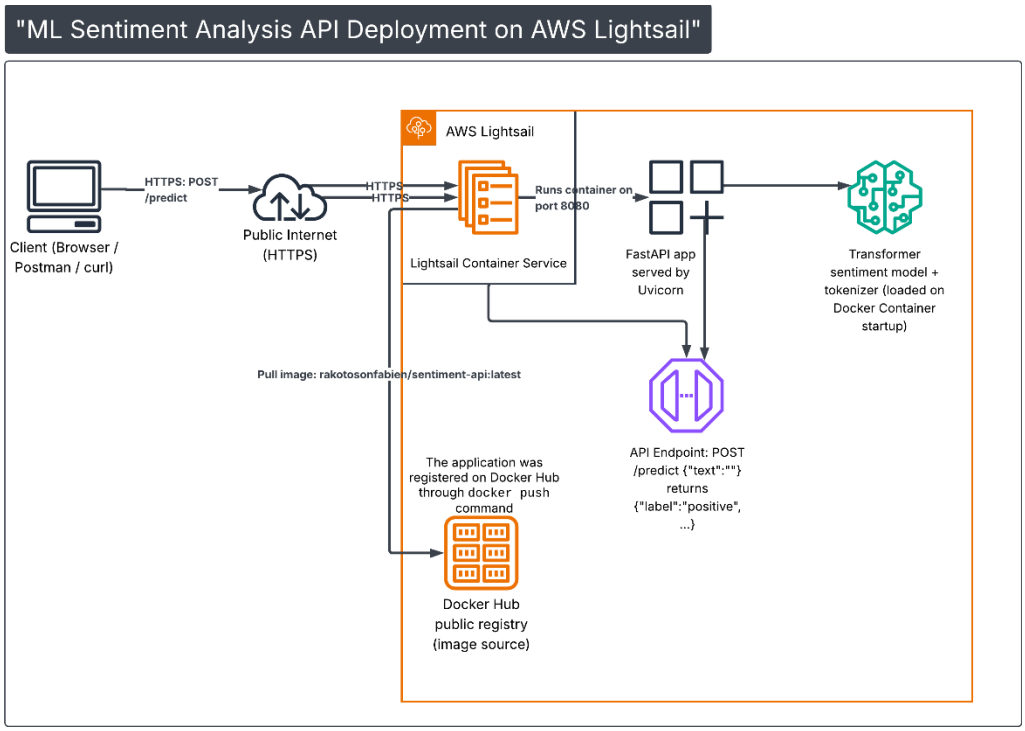


Figure 7: API Deployment Architecture Diagram

Challenges faced

The transformer model increases image size and startup time, so the first deployment on LightSail failed. It could be solved by increasing the timeout/thresholds for the health checks on first startup.

Registering the repository to Docker Hub was a failure because I didn't know I had to log into Docker Hub, create an account and then refer/add my Docker username to the tag to make the push work as expected.

Conclusion

In this project, I trained and evaluated two sentiment models: a TF-IDF + Logistic Regression baseline and a fine-tuned DistilBERT model. The DistilBERT model performed better, so it was selected for deployment.

I packaged the FastAPI app + model into a Docker image, pushed it to Docker Hub, and deployed it on AWS Lightsail Container Service. At the end, the API is live and can be called remotely to get sentiment predictions.

Appendix

- Live Testing command

curl -X POST "https://aig-capstone.jgcz2b6j32c6t.ca-central-1.cs.amazonlightsail.com/predict" -H "Content-Type: application/json" -H "X-API-Key: FABIENRAKOTOSON" -d "{\"text\": \"I was laughing everywhere, that was so funny! \"}"